

Stage-Aware AI Usage and AI Usage Maturity in Software Engineering Education

Anonymous Author1

anon1@university.edu

Anon Institution

Anon City, Anon State, Anon Country

Anonymous Author3

anon3@university.edu

Anon Institution

Anon City, Anon State, Anon Country

Anonymous Author2

anon2@university.edu

Anon Institution

Anon City, Anon State, Anon Country

Anonymous Author4

anon4@university.edu

Anon Institution

Anon City, Anon State, Anon Country

Abstract

Generative artificial intelligence (GENAI) tools, including software that can write code, answer questions, and assist with technical tasks, are rapidly being adopted in software engineering education, yet most empirical studies treat AI adoption as a single, uniform behavior. This paper presents an empirical analysis of how students use and engage with AI across the different phases of software development in an undergraduate software engineering course ($N = 85$). We evaluate two constructs: AI Usage (AU), operationalizing TAM's actual use construct as the frequency with which students engage with GENAI tools, and AI Usage Maturity (AUM), meaning how thoughtfully and deliberately they use it. We examine these across six phases of the software development lifecycle: planning, design, implementation, testing, deployment, and maintenance. We also examine how these relate to students' perceptions using the Technology Acceptance Model (TAM).

Our results show that overall AU and AUM are strongly correlated ($r = 0.690$, $p < .001$), and that AUM differs significantly across development phases (Friedman $\chi^2 = 66.13$, $p < .001$). Students show substantially higher maturity in early and middle phases than in later ones. By contrast, how often students use AI does not vary significantly across phases, meaning usage frequency is relatively uniform while the quality of AI interaction changes meaningfully. The relationship between frequency and maturity is strongest during planning ($r = 0.671$, $p < .001$) and absent during maintenance ($r = 0.164$, n.s.). These findings offer practical guidance for educators seeking to integrate AI more effectively across the full software development process.

CCS Concepts

• **Social and professional topics** → **Computing education**; *Computational thinking*.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGCSE Virtual 2026, Virtual

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

Keywords

generative AI, software engineering education, AI usage maturity, technology acceptance model, software development lifecycle, AI literacy

ACM Reference Format:

Anonymous Author1, Anonymous Author2, Anonymous Author3, and Anonymous Author4. 2026. Stage-Aware AI Usage and AI Usage Maturity in Software Engineering Education. In *Proceedings of ACM Virtual SIGCSE (SIGCSE Virtual 2026)*. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

GenAI tools have fundamentally changed the landscape of software engineering (SE) education. These tools generate code and debug; research shows they reduce cognitive load and increase engagement [3, 4, 9]. At the same time, they introduce important pedagogical concerns: AI generated outputs may be incorrect or adopted uncritically, and overreliance without evaluation may undermine deeper learning [12, 14, 18].

As a result, the central question for educators has shifted from *whether* students use GenAI to *how* they use it. Much prior empirical work models AI adoption primarily as a unitary outcome, typically behavioral intention, acceptance, or use, rather than as a multidimensional set of practices [1, 2, 20]. This abstraction is limiting in software engineering education, where student work unfolds across the Software Development Lifecycle (SDLC)[16]—planning, design, implementation, testing, deployment, and maintenance. Each phase makes different cognitive and technical demands, and understanding how AI use varies across them is essential for curriculum design [17].

In this paper we build on the Technology Acceptance Model (TAM) [5, 6] to develop a Stage-Aware TAM framework, evaluating TAM constructs independently for each SDLC stage. We further introduce AI Usage Maturity (AUM) to capture not merely whether students use AI, but how, specifically whether they engage in iterative prompting, output verification, problem decomposition, and contextual alignment. We address the following research questions:

RQ1 How does AI usage relate to AI usage maturity across SDLC stages?

RQ2 How does AI usage maturity vary across SDLC stages?

2 Background and Related Work

2.1 Technology Acceptance Model

TAM proposes that perceived ease of use (PEOU) and perceived usefulness (PU) jointly predict behavioral intention (BI) and actual use (AU) [5, 6]. This causal chain has been validated extensively in educational technology research, and recent work on AI-based coding assistants confirms PU and PEOU remain central predictors of student adoption [3, 7, 19].

Contemporary TAM extensions incorporate context-specific antecedents including trust, motivation, and facilitating conditions [3, 8]. These extensions recognize that whether a student adopts a technology depends not only on their personal perceptions but also on broader factors such as institutional support, access to resources, and their confidence in the reliability of the tool. In the context of generative AI tools, trust is particularly important because students must judge not only whether a tool is easy and useful, but whether its outputs can be relied upon [3, 8].

In software engineering education specifically, TAM provides a useful lens for understanding why students choose to engage with AI tools at different points in their coursework [7, 19]. However, the standard TAM framework treats adoption as a single, uniform behavior, assuming that a student either accepts or rejects a technology in general. This assumption becomes limiting when students engage with the same tool across qualitatively different tasks, as is the case across the six phases of the SDLC. Our Stage-Aware TAM framework, described in Section 3, addresses this limitation by evaluating TAM constructs independently for each SDLC stage [5, 6].

2.2 AI Adoption in Software Engineering Education

AI adoption has accelerated through GenAI tools which support code generation, explanation, debugging, and rapid iteration [3, 4, 9]. Students generally perceive these tools as helpful for productivity, though concerns remain about reliability, learning outcomes, and overreliance [14, 18]. A key limitation of existing work is its global treatment of AI adoption: most studies measure usage at overall frequency without considering how it varies across the SDLC [17]. A stage-aware approach is both conceptually motivated and practically useful for curriculum design.

2.3 AI Literacy

AI literacy, which encompasses technical proficiency, critical evaluation, and ethical competence, has emerged as a critical moderator of how students engage with GENAI systems [10]. AI literacy is defined as competencies for critically evaluating outputs, communicating with tools, and applying AI responsibly [11, 13]. In educational settings, these competencies are particularly important because students must assess not only whether AI responses are correct, but also whether they are sufficiently detailed and appropriately contextualized for learning [12]. In this study, AI literacy is treated as a global antecedent shaping students' perceptions and interaction practices across all SDLC stages.

2.4 Project-Based SE Courses as a Setting for Stage-Aware Analysis

Project-based software engineering courses provide a natural setting for examining stage-aware AI adoption because they require students to engage sequentially with multiple SDLC phases over a full academic term, rather than in isolated programming exercises. Raibulet and Arcelli Fontana [15] describe project-based SE education as a continuously evolving pedagogical context in which instructors must adapt course content to incorporate emerging technologies, including large language models and generative AI tools, while preserving coverage of core SDLC activities from requirements through maintenance. Their analysis of a multi-year project-based SE course highlights that students encounter genuinely different cognitive and technical demands at each stage, and that effective course design must account for this stage-dependent structure. Our study operationalizes exactly this insight: by measuring TAM constructs and maturity per stage, we observe how AI adoption patterns differ across the same SDLC phases that Raibulet and Arcelli Fontana argue must each receive dedicated pedagogical attention. The course examined in this study follows exactly this structure, requiring student teams to progress through all six SDLC phases over a full academic term at a public research university.

3 Theoretical Model and Constructs

3.1 Stage-Aware TAM

The original Technology Acceptance Model treats adoption as a context-free behavior: a user either accepts or rejects a system [5, 6]. This treatment is adequate for single-task systems, but becomes misleading when applied to a tool engaged across qualitatively different activities. The same student may find GENAI highly useful for planning a software architecture and barely useful for writing a Dockerfile, or vice versa. A single global measure of Perceived Usefulness conflates these fundamentally different judgments.

Our Stage-Aware TAM addresses this by evaluating TAM constructs independently for each SDLC stage (Planning, Design, Implementation, Testing, Deployment, Maintenance), modeling $PEOU_s \rightarrow PU_s \rightarrow BI_s \rightarrow AU_s$ per stage. This assumes perceived usefulness is task-dependent: PU_s reflects whether a student judges AI useful *for the work of stage s*, not whether AI is useful in general. Second, that the $PU \rightarrow AU$ pathway may have different strength at different stages, a prediction we test directly in Section 5. Stages where $PU \rightarrow AU$ is strong indicate that utility judgments drive usage; stages where it is weak indicate that usage is decoupled from judgments of usefulness and is driven instead by habit, task structure, or convenience.

Behavioral intention (BI) was collected only for Planning; the remaining stages are evaluated through the $PU \rightarrow AU$ link alone, a tradeoff between survey length and theoretical completeness.

3.2 AI Usage Maturity (AUM)

While TAM explains adoption, it doesn't describe usage quality—a central question for GENAI: two students with identical PU, PEOU, and AU scores may interact with AI in radically different ways: one copies generated code unchanged, the other iterates on prompts, verifies outputs, and decomposes problems before asking for help.

These are differences in *quality* of engagement, not adoption. To capture this we introduce AI Usage Maturity (AUM) as a construct orthogonal to AU.

AUM is defined as the degree to which students employ structured, iterative and verification-oriented practices when interacting with AI systems. We operationalize AUM along three dimensions, each grounded in established theoretical frameworks: *iterative prompting and refinement* (monitoring and adaptation based on feedback [21]), *output evaluation and verification* (critical evaluation central to AI literacy [11]), and *contextual alignment* (grounding tool use in project constraints). Together, these dimensions capture complementary cognitive and evaluative practices that distinguish deliberate from passive AI engagement.

We operationalize AUM along three dimensions grounded in established theoretical frameworks:

- *Iterative prompting and refinement*
- *Output evaluation and verification*
- *Contextual alignment*

For each stage s , AUM_s is the mean of three items operationalizing these three dimensions in stage-appropriate language:

$$AUM_s = \frac{1}{3} \sum_{i=1}^3 D_{is}$$

Item wording varies by stage to reflect the kind of work students actually do at each phase. For example, at Implementation a verification item asks about code review and testing, while at Deployment the parallel item asks about continuous integration (CI) results and staging logs. Full item texts are available at the anonymized project repository (link omitted for double-blind review).

3.3 Relating AU, AUM, and TAM

The Stage-Aware TAM and AUM constructs answer complementary questions. TAM answers *why* a student uses AI at stage s (because they find it easy and useful) and *how much* (AU_s). AUM answers *how well* they use it. The conceptual model underlying our analysis is that these are separable: a student may have high PU and high AU at a stage while having low AUM, or (less commonly) the reverse. Correlations between AU and AUM therefore become an empirical question rather than a tautology, and the stage-level pattern of those correlations which we report in Section 5 is informative about whether usage frequency is a reliable proxy for usage quality at a given lifecycle stage. Where AU and AUM are tightly coupled, instructors can use observable usage as a signal of engagement quality; where they are decoupled, direct measurement or instruction on interaction practices is needed.

4 Methods

4.1 Participants and Context

This study was conducted in two undergraduate software engineering courses at a research university (anonymized for review). Students work in teams to design, implement, test, and deploy software systems over a full academic term, engaging with all six SDLC stages considered here. Survey responses were collected via Qualtrics.

After data cleaning, $N = 85$ valid responses remained.

The sample is experienced and AI-fluent, which bounds interpretation of the findings. Eighty percent of respondents report three or more years of programming experience and 29% report five or more years; only 19% report fewer than three years. With respect to GENAI use, 89% report using these tools at least weekly (41% daily, 46% weekly), and only 8% report rare or monthly use. Primary tools were Claude (42%) and ChatGPT (34%). This cohort has cleared the initial adoption threshold; variation reflects how experienced users engage, not whether they adopt.

4.2 Differential Response Rates Across Stages

Planning and Implementation sections were completed by all 85 respondents, while the four remaining stages had substantially higher non-response (Table 1). Only 35 respondents (41%) completed all six stages. To assess whether non-response was systematic, we compared Planning-stage AU and AUM between completers and non-completers for each later stage. No significant differences were found (all $p > .10$), consistent with Missing Completely At Random (MCAR). Complete-case analyses use $N = 35$ for AU and $N = 48$ for AUM within-subject comparisons.

Table 1: Effective sample sizes by SDLC stage.

Stage	N (AU)	N (AUM)
Planning	85	85
Design	54	64
Implementation	85	85
Testing	49	59
Deployment	53	61
Maintenance	58	64

4.3 Survey Instrument and Construct Derivation

The survey used a 5-point Likert scale (1 = Strongly Disagree, 5 = Strongly Agree). For each SDLC stage, items were collected for PEOU, PU, and AU (plus BI for Planning only). AUM was measured via three stage-specific items per stage covering iterative prompting, output evaluation, and verification. Stage-specific AU was operationalized as the single TAM item for each stage (TAM_4 for Planning, TAM_3 for all others). AI Literacy and Facilitating Conditions were each computed as row-wise means of their respective global items. Full item text and provenance are available at the anonymized project repository (link omitted for double-blind review).

4.4 Data Analysis

All analysis was conducted in Python using `pandas`, `scipy.stats`, and `numpy`. We report descriptive statistics, Pearson correlations, Cronbach's α for AUM reliability, Friedman tests for within-subject stage comparisons on the complete-case subsample ($N = 35$), and Wilcoxon signed-rank post-hoc tests with Bonferroni correction where applicable.

5 Results

5.1 AUM Reliability

Internal consistency of the three-item AUM scale was acceptable to excellent across stages: Planning ($\alpha = .897$), Testing ($\alpha = .852$), Maintenance ($\alpha = .870$), Implementation ($\alpha = .699$), Design ($\alpha = .669$), and Deployment ($\alpha = .665$). All scales exceed the commonly applied threshold of $\alpha \geq .60$ for exploratory composite scoring, supporting their use as reliable stage-specific measures. The variation in α across stages is instructive: stages where student engagement with AI is likely to follow a consistent workflow (e.g., planning and maintenance) show higher inter-item consistency, while stages where engagement may be more opportunistic (design, deployment) show somewhat lower coherence. This pattern is consistent with the interpretation that AUM captures meaningful, lived differences in how students interact with AI tools rather than a single uniform construct.

5.2 AU and AUM Across SDLC Stages (RQ2)

Table 2 presents descriptive statistics for AU and AUM by stage. Figure 1 overlays the mean profiles of both constructs across the lifecycle.

Table 2: Descriptive statistics for AU and AUM by SDLC stage.

Stage	AU			AUM		
	<i>M</i>	<i>SD</i>	<i>N</i>	<i>M</i>	<i>SD</i>	
Planning	3.19	1.42	85	3.73	1.13	
Design	3.20	0.66	54	3.50	0.52	
Implementation	3.39	1.24	85	3.84	0.97	
Testing	2.98	0.72	49	3.25	0.68	
Deployment	2.96	0.65	53	3.15	0.55	
Maintenance	3.09	0.68	58	3.22	0.62	

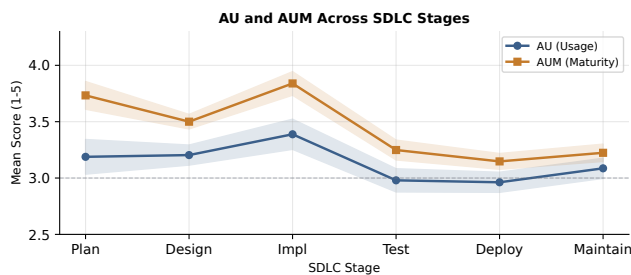


Figure 1: Mean AU and AUM (with ± 1 SE) across SDLC stages. AUM drops after Implementation while AU remains flat.

5.2.1 AUM varies significantly across stages. A Friedman test on the complete-case subsample for AUM ($N = 48$) reveals a strong main effect of stage: $\chi^2(5) = 66.13$, $p < .001$. Post-hoc Wilcoxon signed-rank tests with Bonferroni correction for 15 pairwise comparisons (Table 3) reveal a clear two-cluster structure. Planning, Design, and Implementation form a high-AUM cluster ($M = 3.50$ – 3.84); Testing,

Deployment, and Maintenance form a low-AUM cluster ($M = 3.15$ – 3.25). Every early/middle vs. late stage comparison except Design vs. Testing is significant after Bonferroni correction ($p < .05$), while no pairwise comparison within either cluster survives correction. Planning and Implementation, in particular, are statistically indistinguishable in AUM ($p = .85$), suggesting that the maturity of AI interaction peaks early and carries through coding activities before dropping sharply at testing.

Table 3: Pairwise Wilcoxon signed-rank post-hoc tests for AUM across SDLC stages ($N = 48$). ** $p < .01$; * $p < .05$; n.s. not significant.

Stage A	Stage B	<i>p</i> _{Bonf}	Sig.
Planning	Design	.026	*
Planning	Implementation	1.00	n.s.
Planning	Testing	.001	**
Planning	Deployment	<.001	**
Planning	Maintenance	<.001	**
Design	Implementation	.090	n.s.
Design	Testing	.143	n.s.
Design	Deployment	.004	**
Design	Maintenance	.005	**
Implementation	Testing	.002	**
Implementation	Deployment	<.001	**
Implementation	Maintenance	<.001	**
Testing	Deployment	1.00	n.s.
Testing	Maintenance	1.00	n.s.
Deployment	Maintenance	1.00	n.s.

5.2.2 AU does not vary significantly across stages. In the same complete-case subsample for AU ($N = 35$), the Friedman test finds *no significant difference in AU across stages*: $\chi^2(5) = 5.04$, $p = .41$. When sample composition is held constant, students report broadly similar AI usage frequency across all six SDLC stages. The contrast with the AUM result is striking: over the same lifecycle and same students, *how they engage with AI varies enormously while how often they engage does not*.

5.3 Overall AU–AUM Relationship (RQ1)

At the aggregate level, overall AU and overall AUM are strongly positively correlated ($r = 0.690$, $p < .001$, $N = 85$), as shown in Figure 2. Students who report more frequent AI use also tend to engage in more mature usage practices.

5.3.1 The AU–AUM coupling is stage-dependent. Figure 3 presents stage-level AU–AUM Pearson correlations. Table 4 reports both AU–AUM and PU–AU correlations by stage. Three stages show a statistically significant positive coupling: Planning ($r = 0.671$, $p < .001$), Implementation ($r = 0.462$, $p < .001$), and Deployment ($r = 0.443$, $p = .001$). Three stages do not: Design ($r = 0.268$, $p = .063$), Testing ($r = 0.278$, $p = .061$), and Maintenance ($r = 0.164$, $p = .224$).

5.4 Stage-Level TAM Pathways: PU $\rightarrow$ AU

While full TAM pathway data are available only for Planning, the stage-specific PU and AU items available for all six stages permit a partial test of whether the PU $\rightarrow$ AU link in TAM holds uniformly

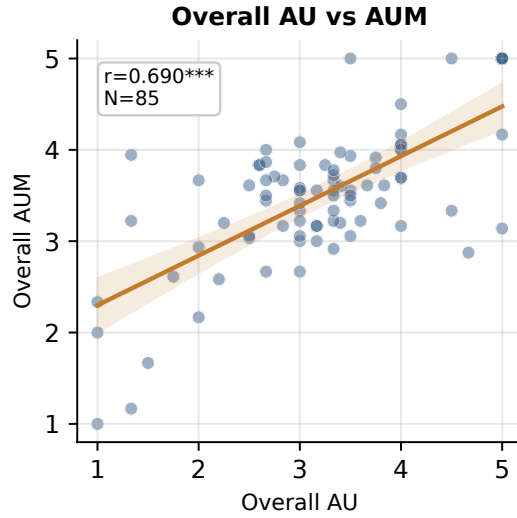


Figure 2: Overall AU vs AUM with regression line and 95% confidence band ($r = 0.690$, $p < .001$, $N = 85$).

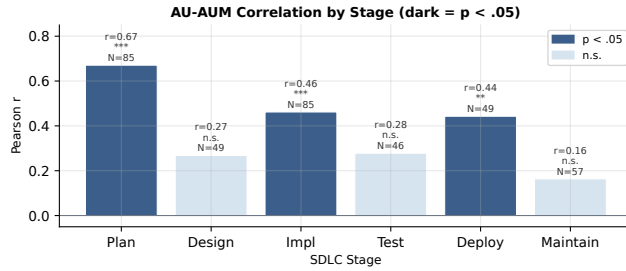


Figure 3: Stage-level AU–AUM correlations. Dark bars: $p < .05$; light bars: n.s. Planning shows strongest coupling.

Table 4: Stage-level correlations: AU–AUM and PU–AU.

Stage	AU–AUM			PU–AU		
	r	p	N	r	p	N
Planning	.671	< .001	85	.839	< .001	85
Design	.268	.063	49	.643	< .001	50
Implementation	.462	< .001	85	.552	< .001	85
Testing	.278	.061	46	.460	.002	43
Deployment	.443	.001	49	.447	.001	49
Maintenance	.164	.224	57	.568	< .001	55

All PU–AU correlations significant at $p < .01$.

Note: Correlation N reflects pairwise deletion for cases missing either variable.

across the SDLC, or whether perceived usefulness predicts usage more tightly at some stages than others. The PU–AU columns in Table 4 show stage-level correlations between perceived usefulness and usage. All six correlations are positive and statistically significant ($p < .01$), confirming that perceived usefulness predicts self-reported usage at every lifecycle stage. The magnitude, however,

varies substantially: PU→AU is very strong at Planning ($r = .84$), moderate through Design ($r = .64$), Implementation ($r = .55$), and Maintenance ($r = .57$), and weaker but still significant at Testing ($r = .46$) and Deployment ($r = .45$).

Because stage-specific AU was operationalized as a single item, the PU–AU correlations in Table 4 should be interpreted as conservative estimates; measurement error in single-item scales attenuates observed correlations, suggesting the true PU→AU Relationships may be somewhat stronger than reported. The attenuation of PU→AU at later stages is theoretically informative. In classical TAM, the PU → AU pathway reflects the degree to which believing a tool is useful translates into actually using it. A weaker PU→AU link at Testing and Deployment suggests that, in those stages, students’ actual AI use is decoupled from how useful they judge AI to be for the task. Together with non-significant AU–AUM coupling, this suggests late-stage AI use is opportunistic and weakly regulated.

5.5 TAM Constructs and Antecedents

Full TAM data (PEOU, PU, BI, AU) are available for the Planning stage. Figure 4 shows the full correlation matrix. TAM intercorrelations are uniformly strong (PU–BI: $r = .88$; PU–AU: $r = .85$; PEOU–PU: $r = .74$), consistent with prior TAM validation studies [3, 19]. While these correlations cannot establish causality, they provide evidence consistent with the theorized TAM pathway. Planning-stage AU correlates strongly with overall AU ($r = .80$), suggesting that early-stage behavior is predictive of lifecycle-wide engagement patterns.

AI Literacy shows a weak relationship with overall AU ($r = .23$, $p = .054$, n.s.) and a stronger relationship with overall AUM ($r = .35$, $p = .004$). Facilitating Conditions shows only a weak relationship with overall AU ($r = .07$, $p = .54$, n.s.). This Asymmetry is theoretically meaningful: in a cohort where tool access is effectively universal, Facilitating Conditions shows near-zero variance in predicting AI use, while AI Literacy, encompassing critical evaluation, ethical competence, and interpreting limitations, more strongly predicts the quality of that use. The remaining variation in mature AI use is better explained by cognitive and evaluative skills than by infrastructure availability. These correlations also provide initial evidence for the discriminant validity of AUM. AUM correlates more strongly with AI Literacy ($r = .30$) than AU does ($r = .23$), and less strongly with PU ($r = .46$) than AU does with PU ($r = .76$), suggesting that AUM captures something meaningfully distinct from both general AI adoption and perceived usefulness—specifically, the critical evaluation and interpretive competence components of AI literacy that usage frequency alone does not reflect. The moderate AU–AUM correlation ($r = .69$) is consistent with two separable constructs that share conceptual overlap but are not redundant: students who use AI more also tend to use it more deliberately, yet meaningful individual-level variation remains at every level of usage frequency.

6 Discussion

6.1 Revisiting the Research Questions

We now return to the two research questions that motivated this study and use the empirical patterns to state what each finding means for practitioners.

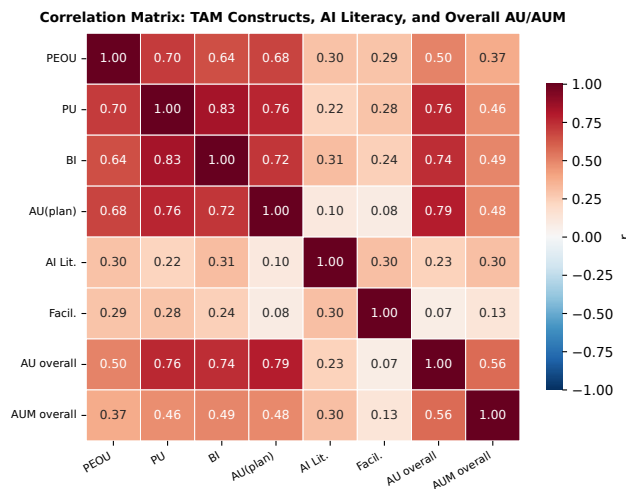


Figure 4: Correlation matrix: Planning-stage TAM constructs, AI Literacy, Facilitating Conditions, and overall AU/AUM.

RQ1: How does AI usage relate to AI usage maturity across SDLC stages? Overall AU and AUM correlate strongly ($r = .69$), but stage-level patterns vary: Three stages show a *significant positive AU–AUM coupling*: Planning ($r = .67$), Implementation ($r = .46$), and Deployment ($r = .44$). In these stages, students who use AI more also use it more maturely. Three stages show *no significant coupling*: Design ($r = .27$), Testing ($r = .28$), and Maintenance ($r = .16$). In these stages, usage frequency tells us nothing about usage quality. The heterogeneity is itself the finding: any educational response must be stage-specific.

RQ2: How does AI usage maturity vary across SDLC stages? Very substantially, and in a structured way. AUM separates cleanly into a high cluster (Planning, Design, Implementation; $M \approx 3.6$) and a low cluster (Testing, Deployment, Maintenance; $M \approx 3.2$), with essentially every cross-cluster pairwise comparison significant after Bonferroni correction (Table 3). Students have, on average, developed disciplined AI practices for the first half of the SDLC and have not yet developed them for the second half. Meanwhile AU is flat across the same stages. To our knowledge, this is among the first empirical results in a computer science (CS) educational setting to show that AI usage quality and AI usage frequency can diverge systematically across the software development lifecycle.

6.2 What Should Instructors Actually Do?

Combining the RQ1 and RQ2 patterns with the stage-level correlations (Table 4), four concrete instructional priorities follow.

1. Seed mature practices during Planning. Planning behavior is the strongest predictor of lifecycle-wide AI engagement ($r = .80$). With the strongest $PU \rightarrow AU$ ($r = .84$) and $AU \rightarrow AUM$ ($r = .67$) couplings, this stage is ideal for intervention. Instructors should use scaffolds like requiring teams to document AI constraints and defend planning outputs before technical design.

2. Change the structure of later-stage AI use, not just its frequency. The low-AUM cluster (Testing, Deployment, Maintenance) shows opportunistic use without clear utility judgments or

structured practices. More AI prompts will raise AU without raising AUM. Instead, use structural scaffolding: require test-generation prompt documentation and acceptance rationale (testing); pair AI-suggested CI/CD changes with staging log review (deployment); frame AI as hypothesis generator requiring verification (maintenance).

3. Prioritize literacy and differentiation. For AI-fluent cohorts, tool access is universal and does not predict maturity (Facilitating Conditions: $r = .07$, n.s.). Instead, focus on direct instruction in critiquing AI outputs—topics that fit naturally into course units on code review and testing. Use tiered scaffolding: structured templates for low-AUM students and critique-focused exercises for high-AUM students, addressing within-cohort heterogeneity that aggregated means do not expose.

7 Limitations and Future Work

This study has important limitations. First, all measures are self-reported, potentially subject to social desirability bias; triangulation with observable artifacts (prompt logs, commit histories) would strengthen validity. Second, stage-specific AU was measured with single items, limiting reliability and potentially attenuating correlations; future work should employ multi-item scales. Third, the sample (single institution, experienced students, AI-fluent cohort) limits generalizability; replications across diverse contexts are needed. Fourth, team-based project structure means responses may not be independent. Fifth, differential non-response across stages (59% incomplete) introduces uncertainty, though MCAR analysis suggests non-systematic attrition.

Future work should link self-reported AU and AUM to code quality metrics via repository mining, examine AUM evolution longitudinally, and test whether pedagogical interventions can improve maturity at late-SDLC stages (testing, deployment, maintenance) where current data show greatest need.

8 Conclusion

This study examines stage-aware GenAI adoption across 85 undergraduate students. Three findings stand out. AI usage frequency does not differ significantly across SDLC stages when sample composition is held constant. AI usage maturity does differ (Friedman $\chi^2 = 66.13$, $p < .001$), with Planning, Design, and Implementation showing meaningfully higher AUM than Testing, Deployment, and Maintenance. The coupling between the two is itself stage-dependent: tightest in Planning ($r = .671$) and absent in Maintenance ($r = .164$, n.s.). Together with stage-level $PU \rightarrow AU$ evidence, these patterns support a stage-aware perspective on AI adoption and provides a practical, correlation-grounded framework for assessing AI-assisted development practices across the entire lifecycle.

Acknowledgments

AI Disclosure. Claude (Anthropic) was utilized to support sections of this work, including writing assistance and data analysis code.

References

- [1] Benicio Gonzalo Acosta-Enriquez, Emma Verónica Ramos Farroñan, Luigi Italo Villena Zapata, Francisco Segundo Mogollon Garcia, Helen Catalina Rabanal-León, Jahaira Eulalia Morales Angaspilco, and Jesús Catherine Saldaña Bocanegra. 2024. Acceptance of Artificial Intelligence in University Contexts: A Conceptual Analysis Based on UTAUT2 Theory. *Heliyon* 10, 19 (2024), e38315. doi:10.1016/j.heliyon.2024.e38315
- [2] Norah Alharbi, Fareed Ud Din, David Paul, and Edmund Sadgrove. 2025. Driving AI Chatbot Adoption: A Systematic Review of Factors, Barriers, and Future Research Directions. *Journal of Open Innovation: Technology, Market, and Complexity* 11, 3 (2025), 100590. doi:10.1016/j.joitmc.2025.100590
- [3] Fawad Ali, Aakash Ahmed, Mohammad Amin Alipour, and Hugo Terashima-Marin. 2025. Adoption of AI-coding Assistants in Programming Education: Exploring Trust and Learning Motivation Through an Extended Technology Acceptance Model. *Journal of Computers in Education* (2025). doi:10.1007/s40692-025-00345-w
- [4] Brett A. Becker, Paul Denny, James Finnie-Ansley, Andrew Luxton-Reilly, James Prather, and Eddie Antonio Santos. 2023. Programming Is Hard—Or at Least It Used to Be: Educational Opportunities and Challenges of AI Code Generation. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education (SIGCSE)*. ACM, 500–506. doi:10.1145/3545945.3569759
- [5] Fred D. Davis. 1989. Perceived Usefulness, Perceived Ease of Use, and User Acceptance of Information Technology. *MIS Quarterly* 13, 3 (1989), 319–340. doi:10.2307/249008
- [6] Fred D. Davis, Richard P. Bagozzi, and Paul R. Warshaw. 1989. User Acceptance of Computer Technology: A Comparison of Two Theoretical Models. *Management Science* 35, 8 (1989), 982–1003. doi:10.1287/mnsc.35.8.982
- [7] Chetna Gupta, Varun Gupta, and Agnieszka Stachowiak. 2021. Adoption of ICT-Based Teaching in Engineering: An Extended Technology Acceptance Model Perspective. *IEEE Access* 9 (2021), 58652–58666. doi:10.1109/ACCESS.2021.3072782
- [8] Fatma Gül Karaoglan Yilmaz, Ramazan Yilmaz, and Mehmet Ceylan. 2023. Generative Artificial Intelligence Acceptance Scale: A Validity and Reliability Study. *International Journal of Human-Computer Interaction* (2023). doi:10.1080/10447318.2023.2283505
- [9] Majeed Kazemitabaar, Justin Chow, Carl Ka To Ma, Barbara J. Ericson, David Weintrop, and Tovi Grossman. 2023. Studying the Effect of AI Code Generators on Supporting Novice Learners in Introductory Programming. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. ACM, 1–23. doi:10.1145/3544548.3580919
- [10] Sanghoon Lee and Gyuyeong Park. 2024. Development and Validation of ChatGPT Literacy Scale. *Current Psychology* 43 (2024), 18992–19004. doi:10.1007/s12144-024-05756-3
- [11] Duri Long and Brian Magerko. 2020. What is AI Literacy? Competencies and Design Considerations. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*. ACM, 1–16. doi:10.1145/3313831.3376727
- [12] Sagnik Nath and So Yoon Yoon. 2024. WIP: Beyond Code: Evaluating ChatGPT, Gemini, Claude, and Meta AI as AI Tutors in Computer Science and Engineering Education. In *Proceedings of the IEEE Frontiers in Education Conference (FIE)*. IEEE, 1–5. doi:10.1109/FIE61694.2024.10893528
- [13] Davy Tsz Kit Ng, Jac Ka Lok Leung, Samuel Kai Wah Chu, and Maggie Shen Qiao. 2021. Conceptualizing AI Literacy: An Exploratory Review. *Computers and Education: Artificial Intelligence* 2 (2021), 100041. doi:10.1016/j.caeai.2021.100041
- [14] Faheem Abbas Pirzado, Aakash Ahmed, Rebeca Abigail Mendoza-Urdiales, and Hugo Terashima-Marin. 2024. Navigating the Pitfalls: Analyzing the Behavior of LLMs as a Coding Assistant for Computer Science Students—A Systematic Review of the Literature. *IEEE Access* (2024). doi:10.1109/ACCESS.2024.3354588
- [15] Claudia Raibulet and Francesca Arcelli Fontana. 2025. Teaching Software Engineering: A Continuously Evolving Perspective. *IEEE Access* 13 (2025), 141038–141063. doi:10.1109/ACCESS.2025.3596845
- [16] Nayan B Ruparelia. 2010. Software development lifecycle models. *ACM SIGSOFT Software Engineering Notes* 35, 3 (2010), 8–13.
- [17] Daniel Russo. 2024. Navigating the Complexity of Generative AI Adoption in Software Engineering. *ACM Transactions on Software Engineering and Methodology* 33, 5 (2024), 135. doi:10.1145/3639472
- [18] Faisal Ahamed Sakib, Saif Hasan Khan, and A. H. M. Rezaul Karim. 2023. Extending the Frontier of ChatGPT: Code Generation and Debugging. arXiv:2307.08260
- [19] Arif Serhan Şimşek, Gülsah Teptaş Cengiz, and Mehmet Bal. 2025. Extending the TAM Framework: Exploring Learning Motivation and Agility in Educational Adoption of Generative AI. *Education and Information Technologies* 30 (2025), 20913–20942. doi:10.1007/s10639-024-13260-7
- [20] Rana Taheri, Neda Nazemi, Sarah E. Pennington, Jason A. Clark, and Faraz Dadgostari. 2025. Factors Influencing Educators' AI Adoption: A Grounded Meta-Analysis Review. *Computers and Education: Artificial Intelligence* (2025). doi:10.1016/j.caeai.2025.100543
- [21] Barry J. Zimmerman. 2002. Becoming a Self-Regulated Learner: An Overview. *Theory into Practice* 41, 2 (2002), 64–70. doi:10.1207/s15430421tip4102_2